T08 – Espaços para capturas do MySQL

T08 – Banco de Dados | Prof. Calvetti

Nome: Eduardo Rafael de Moraes Juvenasso. RA:825135809

Exercício Proposto 1 – INNER JOIN e autorrelacionamento

1. Pedidos com nome do cliente e status

```
34      -- 1. Pedidos com nome do cliente e status
35 •    SELECT p.id_pedido, c.nome AS cliente, p.status
36      FROM pedido p INNER JOIN cliente c ON c.id_cliente = p.id_cliente
37      ORDER BY p.id_pedido;
38
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	cliente	status
101	Ana Lima	PAGO
102	Ana Lima	PAGO
103	Ana Lima	PAGO
104	Bruno Costa	PENDENTE
105	Bruno Costa	PAGO
106	Carla Souza	CANCELADO
107	Diego Alves	PAGO

2. Produtos e subtotais de cada item

```
39      -- 2. Produtos e subtotais de cada item
40  •   SELECT i.id_pedido, pr.nome AS produto, i.quantidade, i.preco_praticado,
41             i.quantidade * i.preco_praticado AS subtotal
42      FROM item_pedido i INNER JOIN produto pr ON pr.id_produto = i.id_produto
43      ORDER BY i.id_pedido, pr.nome;
44
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	produto	quantidade	preco_praticado	subtotal
101	Caderno Universitário	2	22.50	45.00
101	Livro SQL Básico	1	65.00	65.00
102	Caneta Azul	3	3.50	10.50
102	Guia de Banco de Dados	1	89.90	89.90
103	Garrafa Térmica	1	45.00	45.00
103	Mochila Acadêmica	1	120.00	120.00
104	Livro SQL Básico	2	65.00	130.00
104	Régua 30 cm	2	8.00	16.00
105	Caderno Universitário	1	22.50	22.50
105	Caneta Azul	5	3.50	17.50
106	Guia de Banco de Dados	1	89.90	89.90
107	Garrafa Térmica	2	45.00	90.00

3. Caminho PEDIDO, ITEM_PEDIDO e PRODUTO

```
45      -- 3. Caminho PEDIDO, ITEM_PEDIDO e PRODUTO
46 •    SELECT p.id_pedido, c.nome AS cliente, p.status, pr.nome AS produto,
47             i.quantidade, i.quantidade * i.preco_praticado AS subtotal
48      FROM pedido p JOIN cliente c ON c.id_cliente = p.id_cliente
49      JOIN item_pedido i ON i.id_pedido = p.id_pedido
50      JOIN produto pr ON pr.id_produto = i.id_produto
51      ORDER BY p.id_pedido, pr.nome;
52
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	cliente	status	produto	quantidade	subtotal
101	Ana Lima	PAGO	Caderno Universitário	2	45.00
101	Ana Lima	PAGO	Livro SQL Básico	1	65.00
102	Ana Lima	PAGO	Caneta Azul	3	10.50
102	Ana Lima	PAGO	Guia de Banco de Dados	1	89.90
103	Ana Lima	PAGO	Garrafa Térmica	1	45.00
103	Ana Lima	PAGO	Mochila Acadêmica	1	120.00
104	Bruno Costa	PENDENTE	Livro SQL Básico	2	130.00
104	Bruno Costa	PENDENTE	Régua 30 cm	2	16.00
105	Bruno Costa	PAGO	Caderno Universitário	1	22.50
105	Bruno Costa	PAGO	Caneta Azul	5	17.50
106	Carla Souza	CANCELA...	Guia de Banco de Dados	1	89.90
107	Diego Alves	PAGO	Garrafa Térmica	2	90.00

4. Empregados e respectivos supervisores

```
53      -- 4. Empregados e respectivos supervisores
54 •    SELECT e.nome AS empregado, s.nome AS supervisor
55      FROM empregado e LEFT JOIN empregado s ON s.id_empregado = e.id_supervisor
56      ORDER BY e.id_empregado;
57
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empregado	supervisor
Mariana Lopes	NULL
Pedro Nunes	Mariana Lopes
Julia Freitas	Mariana Lopes
Rafael Gomes	Pedro Nunes
Camila Prado	Pedro Nunes

5. Justificativa das condições ON

As condições ON relacionam as chaves corretas: pedido-cliente, pedido-item e item-produto. No autorrelacionamento, os aliases e e s representam empregado e supervisor.

Exercício Proposto 2 – Junções externas e ausência

1. Todos os clientes, inclusive sem pedidos

```
63      -- 1. Todos os clientes, inclusive sem pedidos
64  •   SELECT c.id_cliente, c.nome, p.id_pedido, p.status
65      FROM cliente c LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
66      ORDER BY c.id_cliente, p.id_pedido;
67
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nome	id_pedido
Ana Lima	101
Ana Lima	102
Ana Lima	103
Bruno Costa	104
Bruno Costa	105
Carla Souza	106
Diego Alves	107
Eva Martins	NULL
Felipe Rocha	NULL

2. Produtos nunca vendidos

```sql
-- 2. Produtos nunca vendidos
SELECT pr.id_produto, pr.nome
FROM produto pr LEFT JOIN item_pedido i ON i.id_produto = pr.id_produto
WHERE i.id_produto IS NULL ORDER BY pr.nome;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome	id_pedido
1	Ana Lima	101
1	Ana Lima	102
1	Ana Lima	103
2	Bruno Costa	104
2	Bruno Costa	105
3	Carla Souza	106
4	Diego Alves	107
5	Eva Martins	NULL
6	Felipe Rocha	NULL

3. RIGHT JOIN equivalente com LEFT JOIN

```sql
-- 3. RIGHT JOIN equivalente com LEFT JOIN
SELECT c.nome, p.id_pedido
FROM cliente c LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
ORDER BY c.nome, p.id_pedido;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

categoria	quantidade_produtos	menor_preco	maior_preco	preco_medio
ACESSORIO	2	45.00	120.00	82.50
LIVRO	3	65.00	110.00	88.30
PAPELARIA	3	3.50	22.50	11.33

4. Simulação de FULL OUTER JOIN

```
78      -- 4. Simulação de FULL OUTER JOIN
79      SELECT c.id_cliente, c.nome, p.id_pedido
80      FROM cliente c LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
81      UNION ALL
82      SELECT c.id_cliente, c.nome, p.id_pedido
83      FROM cliente c RIGHT JOIN pedido p ON p.id_cliente = c.id_cliente
84      WHERE c.id_cliente IS NULL;
85
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome	total_pedidos
1	Ana Lima	3
2	Bruno Costa	2
3	Carla Souza	1
4	Diego Alves	1
5	Eva Martins	0
6	Felipe Rocha	0

5. Por que WHERE pode eliminar linhas preservadas?

No LEFT JOIN, clientes sem pedidos ficam com colunas de pedido em NULL. Um filtro no WHERE sobre essas colunas pode removê-los. Para preservar todos os clientes e limitar os pedidos relacionados, o filtro pode ficar no ON.

Exercício Proposto 3 – GROUP BY e HAVING

1. Quantidade, menor, maior e média por categoria

```
91      -- 1. Quantidade, menor, maior e média por categoria
92 •    SELECT categoria, COUNT(*) AS quantidade_produtos, MIN(preco) AS menor_preco,
93             MAX(preco) AS maior_preco, ROUND(AVG(preco),2) AS preco_medio
94      FROM produto GROUP BY categoria ORDER BY categoria;
95
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

categoria	quantidade_produtos	menor_preco	maior_preco	preco_medio
ACESSORIO	2	45.00	120.00	82.50
LIVRO	3	65.00	110.00	88.30
PAPELARIA	3	3.50	22.50	11.33

2. Pedidos por cliente, incluindo zero

```
96      -- 2. Pedidos por cliente, incluindo zero
97 •    SELECT c.id_cliente, c.nome, COUNT(p.id_pedido) AS total_pedidos
98      FROM cliente c LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
99      GROUP BY c.id_cliente, c.nome ORDER BY total_pedidos DESC, c.nome;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome	total_pedidos
1	Ana Lima	3
2	Bruno Costa	2
3	Carla Souza	1
4	Diego Alves	1
5	Eva Martins	0
6	Felipe Rocha	0

3. Clientes com pelo menos dois pedidos

```sql
-- 3. Clientes com pelo menos dois pedidos
SELECT c.id_cliente, c.nome, COUNT(p.id_pedido) AS total_pedidos
FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome HAVING COUNT(p.id_pedido) >= 2
ORDER BY total_pedidos DESC, c.nome;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome	total_pedidos
1	Ana Lima	3
2	Bruno Costa	2

4. Valor total por pedido

```sql
-- 4. Valor total por pedido
SELECT i.id_pedido, SUM(i.quantidade * i.preco_praticado) AS valor_total
FROM item_pedido i GROUP BY i.id_pedido ORDER BY i.id_pedido;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	valor_total
101	110.00
102	100.40
103	165.00
104	146.00
105	40.00
106	89.90
107	90.00

5. Justificativa de WHERE, GROUP BY e HAVING

WHERE filtra linhas antes do agrupamento; GROUP BY define os grupos; HAVING filtra os grupos depois das agregações. COUNT(p.id_pedido) conta apenas pedidos reais, inclusive retornando zero para clientes sem pedidos no LEFT JOIN.

Desafio Integrador – oito consultas

1. INNER JOIN: pedidos, clientes e status

```
116     -- 1. INNER JOIN: pedidos, clientes e status
117 •   SELECT p.id_pedido, c.id_cliente, c.nome AS cliente, p.status
118     FROM pedido p INNER JOIN cliente c ON c.id_cliente = p.id_cliente ORDER BY p.id_pedido;
119
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	id_cliente	cliente	status
101	1	Ana Lima	PAGO
102	1	Ana Lima	PAGO
103	1	Ana Lima	PAGO
104	2	Bruno Costa	PENDENTE
105	2	Bruno Costa	PAGO
106	3	Carla Souza	CANCELADO
107	4	Diego Alves	PAGO

2. INNER JOIN: produtos e subtotais

```
120     -- 2. INNER JOIN: produtos e subtotais
121 •   SELECT i.id_pedido, pr.id_produto, pr.nome AS produto, i.quantidade,
122             i.quantidade * i.preco_praticado AS subtotal
123     FROM item_pedido i INNER JOIN produto pr ON pr.id_produto = i.id_produto
124     ORDER BY i.id_pedido, pr.id_produto;
125
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	id_produto	produto	quantidade	subtotal
101	1	Livro SQL Básico	1	65.00
101	3	Caderno Universitário	2	45.00
102	2	Guia de Banco de Dados	1	89.90
102	4	Caneta Azul	3	10.50
103	5	Mochila Acadêmica	1	120.00
103	8	Garrafa Térmica	1	45.00
104	1	Livro SQL Básico	2	130.00
104	6	Régua 30 cm	2	16.00
105	3	Caderno Universitário	1	22.50
105	4	Caneta Azul	5	17.50
106	2	Guia de Banco de Dados	1	89.90
107	8	Garrafa Térmica	2	90.00

3. LEFT JOIN: clientes com ou sem pedidos

```
126     -- 3. LEFT JOIN: clientes com ou sem pedidos
127 •   SELECT c.id_cliente, c.nome, p.id_pedido, p.status
128     FROM cliente c LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
129     ORDER BY c.id_cliente, p.id_pedido;
130
131     -- 4. Ausência: produtos nunca vendidos
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome	id_pedido	status
1	Ana Lima	101	PAGO
1	Ana Lima	102	PAGO
1	Ana Lima	103	PAGO
2	Bruno Costa	104	PENDENTE
2	Bruno Costa	105	PAGO
3	Carla Souza	106	CANCELADO
4	Diego Alves	107	PAGO
5	Eva Martins	NULL	NULL
6	Felipe Rocha	NULL	NULL

4. Ausência: produtos nunca vendidos

```
131     -- 4. Ausência: produtos nunca vendidos
132 •   SELECT pr.id_produto, pr.nome
133     FROM produto pr LEFT JOIN item_pedido i ON i.id_produto = pr.id_produto
134     WHERE i.id_produto IS NULL ORDER BY pr.nome;
135
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_produto	nome
7	Livro MySQL Avançado

5. Autorrelacionamento: empregado e supervisor

```sql
-- 5. Autorrelacionamento: empregado e supervisor
SELECT e.id_empregado, e.nome AS empregado, s.nome AS supervisor
FROM empregado e LEFT JOIN empregado s ON s.id_empregado = e.id_supervisor
ORDER BY e.id_empregado;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_empregado	empregado	supervisor
1	Mariana Lopes	NULL
2	Pedro Nunes	Mariana Lopes
3	Julia Freitas	Mariana Lopes
4	Rafael Gomes	Pedro Nunes
5	Camila Prado	Pedro Nunes

6. LEFT JOIN agregado: pedidos por cliente, incluindo zero

```sql
-- 6. LEFT JOIN agregado: pedidos por cliente, incluindo zero
SELECT c.id_cliente, c.nome, COUNT(p.id_pedido) AS total_pedidos
FROM cliente c LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome ORDER BY c.id_cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome	total_pedidos
1	Ana Lima	3
2	Bruno Costa	2
3	Carla Souza	1
4	Diego Alves	1
5	Eva Martins	0
6	Felipe Rocha	0

7. LEFT JOIN agregado: total de itens por cliente, incluindo zero

```
146    -- 7. LEFT JOIN agregado: total de itens por cliente, incluindo zero
147    SELECT c.id_cliente, c.nome, COALESCE(SUM(i.quantidade * i.preco_praticado),0) AS total_itens
148    FROM cliente c LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
149    LEFT JOIN item_pedido i ON i.id_pedido = p.id_pedido
150    GROUP BY c.id_cliente, c.nome ORDER BY total_itens DESC, c.nome;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome	total_itens
1	Ana Lima	375.40
2	Bruno Costa	186.00
4	Diego Alves	90.00
3	Carla Souza	89.90
5	Eva Martins	0.00
6	Felipe Rocha	0.00

8. WHERE + HAVING: clientes com dois pedidos pagos

[COLE A PRINT AQUI]

```
152    -- 8. WHERE + HAVING: clientes com dois pedidos pagos
153    SELECT c.id_cliente, c.nome, COUNT(p.id_pedido) AS pedidos_pagos
154    FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente
155    WHERE p.status = 'PAGO'
156    GROUP BY c.id_cliente, c.nome HAVING COUNT(p.id_pedido) >= 2
157    ORDER BY pedidos_pagos DESC, c.nome;
158
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	nome	pedidos_pagos
1	Ana Lima	3